



ABF-RRT*: Bidirectional fast optimal rapidly-exploring random tree via adaptive sampling of a permissible hyper-ellipsoid

Zhaoyang Zeng¹ · Yufang Chang¹

Received: 2 May 2025 / Accepted: 15 July 2025 / Published online: 18 August 2025
© The Author(s) 2025

Abstract

This paper proposes a global path planning algorithm based on adaptive sampling and bidirectional fast exploration, ABF-RRT*. ABF-RRT* introduces an adaptive sampler that dynamically constructs a hyper-ellipsoid according to the progress of dual-tree exploration and adaptively weighs the sampling inside and outside the specified area, which effectively guides the process of dual trees advancing each other. The improved bidirectional planner combines the bisection optimization process, significantly improves the topological structure of the random tree, and accelerates the convergence to the optimal path solution. Since the algorithm relies on the tree expansion framework, it is compatible with various samplers and graph pruning strategies. Through extensive simulation experiments, the superiority of the two cores is analyzed separately, and it is proved that ABF-RRT* has excellent performance and efficiency in path planning in various environments.

Keywords Motion planning · Sampling-based algorithms · Rapidly-exploring random tree (RRT) · Optimal path planning

1 Introduction

The rapid advancement of robotics has propelled the quest for efficient, safe, and optimized motion planning in complex environments to the forefront of research. Therefore, motion planning algorithms have been widely used in various fields, including but not limited to robot navigation, industrial automation, satellite trajectory planning, and surgical robotics (LaValle 2006; Siciliano & Khatib 2016; Sidi 1997; Xing et al. 2025a, b).

In practice, sampling-based planning algorithms represented by rapid search random trees (RRT) (LaValle 1998) show good robustness and adaptability in high-dimensional space. The RRT algorithm extracts spatial information by randomly sampling the unknown space and explores the spatial state through incremental growth of the tree. After continuous iteration of random samples, the planner gradually constructs a set of trajectories. As the number of sampling points increases, the random tree can describe the structure of the unknown space with higher resolution. Regardless

of the size of the extended dimension, when the number of iterations approaches infinity, the probability of finding a solution is close to 100%. Therefore, RRT has great flexibility in dealing with high-dimensional nonlinear problems and is widely used in robotic systems with complex degrees of freedom and multiple constraints, including driverless cars, robotic arms, drones, ground vehicles, and surgical robots (Boscariol et al. 2015; Yoshikawa & Arimoto 1994; Huang et al. 2023; Lin et al. 2022; Xing et al. 2025a, b). However, the RRT algorithm is only probabilistically complete, with low planning efficiency and unable to obtain the optimal solution for the path (Wang et al. 2022).

As one of the optimal variants of RRT, RRT* has asymptotic optimality (Karaman & Frazzoli 2011). When the number of iterations approaches infinity, the probability of RRT* finding the optimal solution is close to 100%. However, the convergence effect of RRT* depends largely on the quality of random sampling. If there is a lack of sufficient sampling in a specific area, the planning efficiency will be significantly reduced. Informed-RRT* (Gammell et al. 2014a) uses the starting point and the target point to construct a hyper-ellipsoid, which effectively constrains the sampling space. Q-RRT* and Fast-RRT* (F-RRT*) (Jeong et al. 2019; Liao et al. 2021) further improve the convergence speed by locally reconstructing the topological structure of the tree. However, due to the lack of appropriate sampling constraints

✉ Yufang Chang
changyf@hbut.edu.cn

¹ Hubei Collaborative Innovation Center for High-Efficiency Utilization of Solar Energy, Hubei University of Technology, Wuhan 430068, China

and extension guidance, this method shows environmental dependence, and its planning efficiency is not enough to cope with path planning problems in complex environments.

RRT-connect (Kuffner & LaValle 2000), as a variant algorithm of RRT, uses a bidirectional exploration strategy to effectively improve the speed of planning the initial solution. During the planning process, the backward tree will act as a heuristic method for the forward tree, thereby guiding the forward tree to grow incrementally and continuously, thereby improving planning efficiency. ARRT-Connect (Li & Chen 2022) adjusts the sampling strategy according to the size of the unexplored area based on the dual-tree exploration, and combines the principal component analysis (PCA) method of RRV (Tahirovic & Ferizbegovic 2018) to enable the tree to explore the narrow channel environment faster. However, neither RRT-connect nor ARRT-Connect can guarantee path optimality. ABQ-RRT* (Zhuo et al. 2024) not only extends RRT* to bidirectional, but also combines the local reconstruction of Q-RRT* and the new cost function, and achieves fast planning and convergence in narrow channels by backtracking parent nodes and local sampling of obstacles. However, this method performs poorly in scenarios other than narrow environments because it relies too much on local sampling of obstacles to plan the tree structure.

When solving the path planning problem, at least three aspects need to be considered: "how to avoid redundant sampling", "how to effectively explore and grow the tree", and "how to quickly converge to the optimal solution based on the initial path solution". In response to these problems, this paper attempts to obtain an algorithm that takes into account both timeliness and optimality, the Adaptive Bidirectional Fast Random Tree* (ABF-RRT*). The main contributions of this paper are as follows:

1. An adaptive sampling strategy for bidirectional exploration trees is designed, which can adaptively adjust the sampling probability inside and outside the hyper-ellipsoid region according to the progress of the two trees advancing each other, effectively balancing the global exploration and local refinement in the planning process.
2. A fast bidirectional exploration strategy combined with bisection reconstruction of the path topology is proposed, which ensures the rapid identification of various environments in the initial planning process and improves the speed of convergence of the plan to the optimal solution.
3. A tree expansion algorithm is proposed, which can be used in combination with any other sampler or planner.

The structure of this paper is as follows. Section 2 defines the problem and reviews existing algorithms. Section 3 introduces the proposed ABF-RRT* algorithm. Section 4 verifies the core method and conducts comparative experiments and analysis with existing algorithms. Section 5 further discusses

the key factors affecting the efficiency of the algorithm and the limitations of the new algorithm. Section 6 summarizes the paper.

2 Background

2.1 Problem definition

Let $\mathcal{X} \subseteq \mathbb{R}^d$ be the configuration space, where $d \in \mathbb{N}, d > 1$, $\mathcal{X}_{obs} \subset \mathcal{X}$ be the obstacle region, $\mathcal{X}_{free} = \text{closure}(\mathcal{X} \setminus \mathcal{X}_{obs})$ be the obstacle-free region, where $X_{start} \in \mathcal{X}_{free}$ is the initial state and $X_{goal} \in \mathcal{X}_{free}$ is the goal state. Therefore $(\mathcal{X}, X_{start}, X_{goal})$ defines a path planning problem. Let the bounded variational continuous function $\sigma : [0, 1] \rightarrow \mathcal{X}$ be a path. If $\forall \tau \in [0, 1], \sigma(\tau) \in \mathcal{X}_{free}$, then the path is collision-free.

Definition 1. Feasible path planning means finding a collision-free feasible path for a given motion planning problem $(\mathcal{X}, X_{start}, X_{goal})$, $\sigma(0) \in X_{start}, \sigma(1) \in X_{goal}$. If no solution is returned, a failure is reported.

Definition 2. Optimal path planning means finding a feasible path with the minimum cost $\text{Cost}(\sigma^*)$ for a given motion planning problem $(\mathcal{X}, X_{start}, X_{goal})$, so that $\text{Cost}(\sigma^*) = \min\{\text{Cost}(\sigma) : \sigma \text{ is feasible}\}$, where $\text{Cost}(\sigma)$ is the cost from X_{start} to X_{goal} along the path σ .

2.2 Related works

Existing path planning algorithms can be roughly divided into four categories: graph-based planning, reaction-based planning, heuristic-based intelligent planning, and random sampling-based planning.

Graph-based search algorithms include the classic Dijkstra algorithm (Dijkstra 1959) and A* algorithm (Hart et al. 1968), the D* algorithm (Stentz 1994), which extends the A* algorithm to handle dynamic obstacles, and the jumping point search (JPS) algorithm (Tong et al. 2021), which further optimizes the search process. However, graph-based algorithms require a complete discretization of the environment, which leads to a significant increase in computational complexity, especially in high-dimensional spaces.

Reaction-based planning is based on the perception of the current state and is often used for local path planning. Representative algorithms include the artificial potential field method (APF) (O. Khatib 1985) and the dynamic window approach (DWA) (D. Fox et al. 1997). The algorithm logic is simple and the calculation efficiency is high, but it is easy to fall into the local optimal value in complex environments (especially concave obstacles in narrow channels).

Intelligent search algorithms based on heuristics, such as genetic algorithms (Alabbadi & Kanan 2023) inspired by biological evolution, ant colony optimization algorithms (Li et al. 2021) using positive feedback mechanisms, and particle swarm optimization algorithms (Shami et al. 2022) based on cooperative principles, have strong adaptability and can adapt to different environmental conditions and constraints through an iterative process. However, these methods also have obvious defects, including high reliance on heuristic strategies, high parameter sensitivity, and exponential growth in resource requirements in high-dimensional environments (Stentz 1994; Botea et al. 2002).

None of the above algorithms can be effectively expanded to high-dimensional state spaces or larger problem spaces, exposing the problem of insufficient scalability.

Sampling-based planning algorithms, represented by rapid search random trees (RRT) (LaValle 1998). However, the RRT algorithm only has probabilistic completeness, low planning efficiency, and cannot obtain the optimal solution for the path (Wang et al. 2022). Previous studies on RRT are mainly based on improved sampling methods and improved tree expansion methods.

The improved sampling method hopes to improve planning efficiency by changing the distribution of sampling points to ensure that the sampling points are concentrated in key areas that must be passed or difficult to pass. Informed-RRT* (Gammell et al. 2014a) proposed an accurate method using direct sampling of ellipsoid subsets. Under strict assumptions (i.e., no obstacles), this direct sampling can converge linearly to the optimal solution. BIT* (Gammell et al. 2014b) combines the efficient ordering of graph search algorithms with the scalability of sampling algorithms. Cloud-RRT* (Kim et al. 2014) uses generalized Voronoi diagrams to initialize the sampling cloud and uses it as a sampling bias to effectively guide the expansion of the tree. P-RRT* (Huang et al. 2023) and PQ-RRT* (Li et al. 2020) construct artificial potential field graphs to obtain new nodes that are closer to expectations. Neural RRT* (Wang et al. 2020) combines convolutional neural networks (CNNs) to generate a network that can predict the probability distribution of the optimal path area, and uses it to guide the sampling process, accelerating the convergence of motion planning. ARRT-Connect (Li & Chen 2022) parallelizes the C-Forest algorithm to obtain samples from a subset of the planning domain, which is defined as a hyperrectangle that defines the hyper-axis. AI-RRT* (Huang et al. 2024) constructs a hybrid sampler that uses prior knowledge of narrow channels to check sampling points, enhancing the speed of asymptotic optimization.

The method based on improved tree expansion mainly guides the direction of tree expansion according to some strategies, thereby enhancing the ability of the algorithm to handle complex environments. Anytime RRT* (Karaman et al. 2011) and Bi-RRT* (Jordan & Perez 2013) use the

current vertex cost plus the estimated cost to the target point as the heuristic cost, and periodically delete some nodes of the tree. However, this estimation method can easily lead to incorrect pruning, especially in the early stages of the algorithm. RRG (Arslan & Tsiotras 2013) prunes the existing graph by adding an LPA*-style (Koenig et al. 2004) bond to each node. RRT*-smart (Islam et al. 2012) prunes tree nodes through a smart sampling strategy. RRV (Tahirovic & Ferizbegovic 2018) guides the tree to quickly pass through narrow obstacles by forming dominant eigenvectors around obstacles. Quick-RRT* (Jeong et al. 2019) expands the candidate set of parent nodes during the local reconstruction of RRT*, improves the utilization of nodes by using the triangle inequality, improves the quality of the initial solution and the speed of convergence to the optimal solution. F-RRT* (Liao et al. 2021) builds on Q-RRT* and uses a bisection method to create intermediate nodes during the exploration process, which can almost immediately obtain the optimal topology in a local area. However, due to the blindness of sampling, unidirectional exploration algorithms are very time-consuming in complex environments (Wang et al. 2022). RRT-connect (Kuffner & LaValle 2000) developed a bidirectional exploration combined with a heuristic strategy to achieve faster planning. ARRT-Connect (Li & Chen 2022) combined RRV and a new sampling strategy to build a bidirectional planner. ABQ-RRT* (Zhuo et al. 2024) further used an adaptive sampling bias strategy to guide tree growth based on ARRT-Connect. Both enable the tree to plan paths in narrow channels faster.

The ABF-RRT* method proposed in this paper combines the improved sampling-based method and the improved tree expansion-based method. This method designs an adaptive sampler for bidirectional exploration, which dynamically adjusts the size of the specified area and the sampling probability inside and outside the specified area according to the exploration progress of the dual tree, instead of simply performing node rejection (such as Informed-RRT*) or pre-sampling according to the obstacle structure (such as ABQ-RRT*), which is different from the algorithm that adjusts the sampling strategy in the above discussion. In addition, ABF-RRT* couples a binary-based optimization strategy and an exploration strategy based on heuristic bidirectional exploration, which not only reduces the blindness of exploration, but also enables the tree to efficiently cope with various complex environments instead of just quickly processing narrow channels.

3 ABF-RRT*

This section explains the details of the proposed ABF-RRT* algorithm, introduces a greedy sampler for bidirectional exploration and a new method for optimizing bidirectional

exploration paths. At the same time, we derive the computational complexity of the new algorithm and analyze the factors affecting the complexity.

3.1 Proposed algorithm

The main idea of ABF-RRT* is based on the observational comparison of the solution efficiency and convergence results of existing algorithms: Firstly, for the planning algorithm that explores the space through incremental growth, its computational complexity increases exponentially with the number of iterations. Therefore, reducing redundant sampling and exploration and improving the utilization of existing nodes can plan the initial solution faster. Secondly, creating parent nodes through triangle inequalities can not only make the planner have a faster convergence rate, but also reduce the resulting path cost.

The mathematical block diagram of ABF-RRT* is shown in Fig. 1. Specifically, after initializing the dual

trees, the sampling parameters and the size and shape of the hyper-ellipsoid are first updated based on the mutual exploration progress of the dual trees. The sampling probability of the sampler inside and outside the specified area is dynamically adjusted according to the sampling parameters to obtain higher quality sampling points. Then, forward tree expansion and heuristic-based reverse tree expansion are performed, and the binary search method is used to create alternative parent nodes to speed up the convergence process until the dual trees are successfully connected or interrupted by obstacles. It should be noted that in order to avoid too many exploration points in the local space caused by heuristic greedy search, the sizes of the two trees will be compared after each successful exploration, and the tree exchange operation will be performed based on this. The pseudo code of the algorithm is shown in Algorithm 1. Compared with the baseline algorithm RRT*, the main improvements correspond to lines 6, 7, and 10 marked in red in the algorithm.

Algorithm 1: ABF-RRT*

Input : $X_{start}, X_{goal}, n, Map, Stepsize, \delta, \alpha, \beta, p_{init}$
Output : T

```

1   $V_a \leftarrow \{X_{start}\}, V_b \leftarrow \{X_{goal}\};$ 
2   $E_a \leftarrow \emptyset, E_b \leftarrow \emptyset;$ 
3   $X_{e,a} \leftarrow X_{start}, X_{e,b} \leftarrow X_{goal};$ 
4   $T_a \leftarrow (V_a, E_a), T_b \leftarrow (V_b, E_b);$ 
5  for  $i = 1$  to  $n$  do
6     $X_{rand} \leftarrow \text{AdaptiveSample}(X_{e,a}, X_{e,b}, X_{start}, X_{goal}, p_{init}, \alpha, \beta);$ 
7     $(X_{new,a}, T_a, flag_a) \leftarrow \text{FastExtend}(X_{rand}, T_a, Stepsize, \delta);$ 
8     $X_{e,a} \leftarrow X_{new,a};$ 
9    if  $flag_a \neq \text{Trapped}$  then
10      $(X_{new,b}, T_b, flag_b) \leftarrow \text{FastConnect}(X_{new,a}, T_b, Stepsize, \delta);$ 
11      $X_{e,b} \leftarrow X_{new,b};$ 
12     if  $flag_b = \text{Reached}$  then
13        $T \leftarrow T_a \cup T_b;$ 
14       for each edge  $(X_{new}, X_{goal}) \in E_b$  do
15         Add  $(X_{new}, X_{goal})$  to  $E_a;$ 
16       end
17     return  $T;$ 
18   end
19 end
20   Swap;
21 end
22 return Fail;

```

Algorithm 2: AdaptiveSample.

Input : $X_{e,a}, X_{e,b}, X_{start}, X_{goal}, Map, p_{init}, \alpha, \beta$
Output : X_{rand}

- 1 $Distance \leftarrow \|X_{e,a} - X_{e,b}\|_2$;
- 2 $DistanceRatio \leftarrow Distance / \|X_{start} - X_{goal}\|_2$;
- 3 $p_{in} \leftarrow p_{init} \cdot (1 - DistanceRatio) + (1 - p_{init}) \cdot DistanceRatio$;
- 4 **if** $rand < p_{in}$ **then**
- 5 $X_{ball} \leftarrow \text{SampleUnitBall}$;
- 6 $X_{center} \leftarrow (X_{e,a} + X_{e,b}) / 2$;
- 7 $D \leftarrow (X_{e,a} - X_{e,b}) / \|X_{e,a} - X_{e,b}\|_2$;
- 8 $Axis_1 \leftarrow Distance + \alpha \cdot \|X_{e,a} - X_{e,b}\|_2$;
- 9 **for** $j \leftarrow 2$ **to** n **do**
- 10 $Axis_j \leftarrow \beta \cdot Axis_1$;
- 11 **end**
- 12 $L \leftarrow \text{diag}(Axis_1, Axis_2, \dots, Axis_n)$;
- 13 $V \leftarrow [1, 0, \dots, 0]^T$;
- 14 $K \leftarrow \text{ProductMatrix}(V, D)$;
- 15 $\theta \leftarrow \arccos(V^T D)$;
- 16 $R_{matrix} \leftarrow I + \sin(\theta)K + (1 - \cos(\theta))K^2$;
- 17 $X_{ellipse} \leftarrow R_{matrix} \cdot L \cdot X_{ball} + X_{center}$;
- 18 $X_{rand} \leftarrow \text{ProjectToSpace}(X_{ellipse}, Map)$;
- 19 **else**
- 20 $X_{rand} \leftarrow \text{SampleUniform}(Map)$;
- 21 **end**
- 22 **return** X_{rand} ;

3.2 Adaptive sampling

ABF-RRT* hopes to balance the exploration of unknown areas and the use of existing areas during the sampling process. The sampler of ABF-RRT* takes into account the sampling inside and outside the specified area, rather than simply rejecting the sampling points outside the specified area, and continuously shrinks the size of the super ellipsoid according to the exploration progress of the double tree. Therefore, the new algorithm can dynamically adjust the sampling density according to the exploration progress of the tree, so that it will not converge slowly due to excessive global expansion, nor will it cause slow planning speed due to limited exploration, thus balancing global expansion and local exploration well.

The pseudo code of the sampler is given in Algorithm 2. Specifically, first, initialize the sampler parameters (lines 1–3), assume that the current expansion points of the forward tree and the backward tree are $X_{e,a}$ and $X_{e,b}$ respectively, and the shrinkage distance ratio is:

$$\mathcal{R} = \frac{\|X_{e,a} - X_{e,b}\|_2}{\|X_{start} - X_{goal}\|_2} \quad (1)$$

Define the probability measure as:

$$P(X_{rand} \in A) = p_{in} \cdot u_{ellipse}(A) + (1 - p_{in}) \cdot u_{global}(A), \forall A \in X \quad (2)$$

Among them, $u_{ellipse}$ is the uniform probability measure in the hyper-ellipsoid, u_{global} is the heuristic measure in the global space (set as uniform measure in this paper), and p_{in} is the dynamic probability parameter. Specifically:

$$p_{in} = p_{init} + (1 - \mathcal{R}) \cdot (1 - p_{init}) \quad (3)$$

Therefore, the dynamic probability parameter p_{in} will decrease as the shrinkage distance ratio increases.

Next, construct the parameterized equation of the dynamic hyper-ellipsoid $\mathcal{E}$:

$$\mathcal{E} : (X - X_{center})^T \Sigma^{-1} (X - X_{center}) \quad (4)$$

Among them, $X_{center} = \frac{X_{e,a} + X_{e,b}}{2}$ is the center of the hyper-ellipsoid, Σ covariance matrix, specifically:

$$\begin{cases} \Sigma = R \cdot L \cdot R^T \\ L = \text{diag}(Axis_1^2, Axis_2^2, \dots, Axis_d^2) \end{cases} \quad (5)$$

Among them, the main axis $Axis_1 = \|X_{e,a} - X_{e,b}\|_2 + \alpha \cdot \text{Stepsize}$, α is the shrinkage parameter, which is used to prevent the super ellipsoid from degenerating into a point when the double trees are too close.

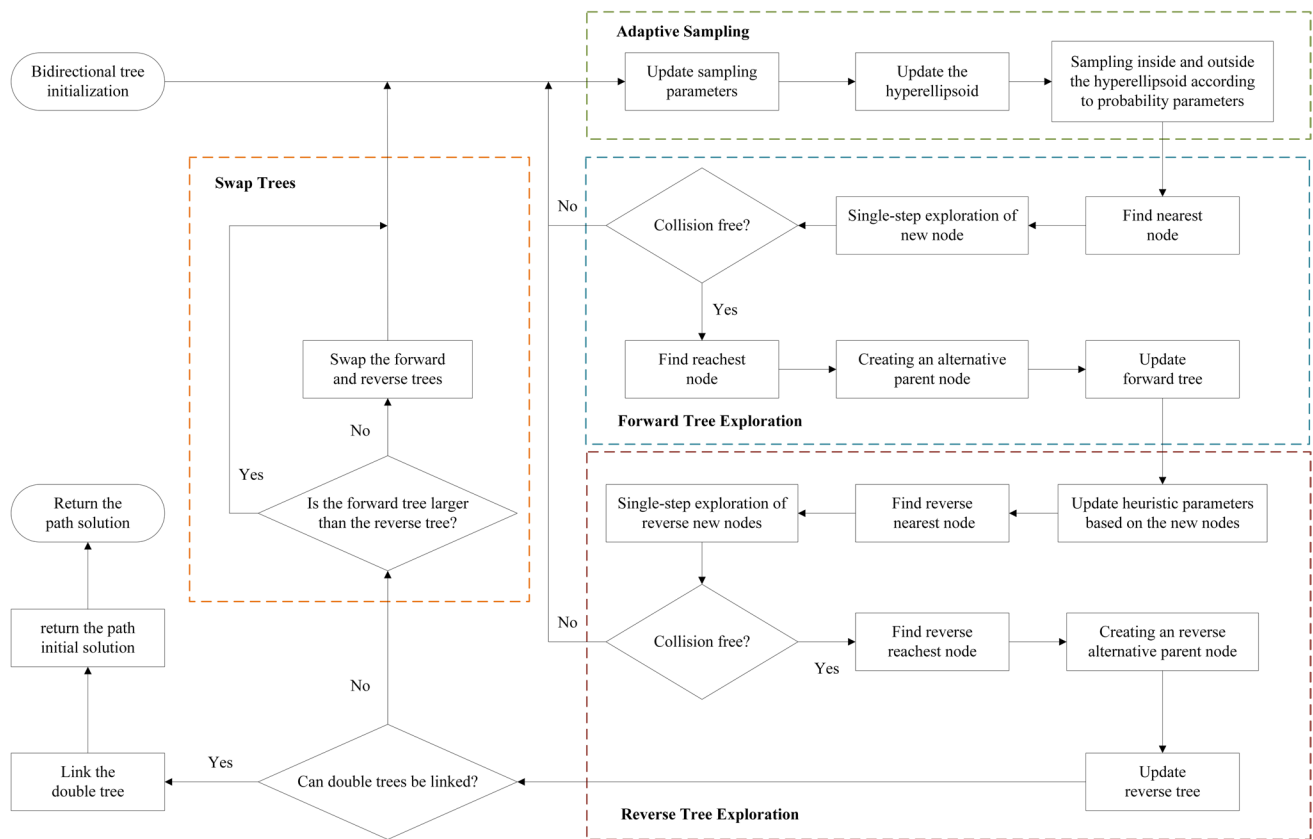


Fig. 1 System mathematical block diagram

The secondary axis $Axis_j = \beta \cdot Axis_1, j \geq 2$, β is the flattening parameter.

The rotation matrix R_{matrix} is obtained by the Rodrigues formula and is used to align the hyper-ellipsoid with the main axis direction. Specifically, assuming the standard basis vector $V = [1, 0, \dots, 0]^T$, the rotation axis is the cross product of the basis vector and the main axis direction D , that is, $R_{axis} = \frac{V \times D}{\|V \times D\|_2}$, assuming $\theta = \arccos(V \cdot D)$ is the rotation angle, then the rotation matrix is:

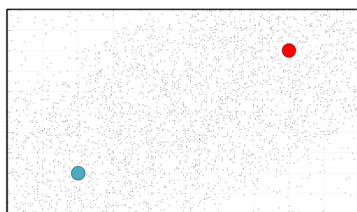
$$R_{matrix} = I + \sin(\theta) \cdot K + (1 - \cos(\theta)) \cdot K^2 \quad (6)$$

Among them, K is the antisymmetric matrix of the rotation axis $R_{axis} = [k_x, k_y, k_z]^T$. In the three-dimensional case:

$$K = \begin{bmatrix} 0 & -k_z & k_y \\ k_z & 0 & -k_x \\ -k_y & k_x & 0 \end{bmatrix}$$

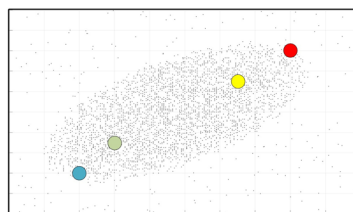
During the actual sampling process, the sampler will randomly generate a random number and compare it (line

At the initial moment, the sampling points are evenly distributed within the ellipse.



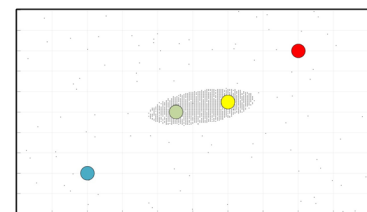
(a) Distance = 85.85

As the double tree exploration point approaches, the area of the ellipse decreases



(b) Distance = 47.70

The probability of sampling within the ellipse increases as the exploration point approaches



(a) Distance = 15.81

Fig. 2 The sampler adaptively adjusts the sampling probability inside and outside the hyper-ellipsoid region according to the exploration progress (blue dots: initial state of the forward tree. Red dots: initial

state of the reverse tree. Green dots: latest state of the forward tree. Yellow dots: latest state of the reverse tree)

4) to decide whether to uniformly sample within the hyper-ellipsoid (lines 5–18) or uniformly sample globally (line 20). Uniform sampling within the hyper-ellipsoid is achieved by transforming the unit sphere into the hyper-ellipsoid coordinate system (lines 13–18). Specifically, first, generate a random point $X_{ball} \in \mathbb{R}^d$ within the hypersphere, and satisfy $\|X_{ball}\|_2 \leq 1$. Then, perform scaling, rotation, and translation operations in sequence:

$$\begin{cases} X_{scaled} = L \cdot X_{ball} \\ X_{rotated} = R_{matrix} \cdot X_{scaled} \\ X_{rand} = X_{rotated} + X_{center} \end{cases} \quad (8)$$

This achieves uniform sampling within the specified hyper-ellipsoid.

Therefore, as the double-tree planning progresses, the dynamic probability parameter will continue to increase, that is, the sampler will be more and more concentrated in the specified super ellipsoid for sampling, and at the same time, the super ellipsoid will continue to shrink. Therefore, in the initial stage of planning, the sampler tends to sample

uniformly in the global range to explore as many unexplored areas as possible. As the double trees gradually approach, the sampling will be more and more concentrated in the area where the double trees may be connected, thereby improving the exploration resolution in the area. Figure 2 shows an example of 5000 adaptive samplings at different double-tree proximity levels.

3.3 Fast exploration

ABF-RRT* applies the strategy of bisection reconstruction of path topology to the heuristic-based bidirectional exploration tree. Although pure bidirectional exploration can eliminate certain blindness by using heuristic strategies, its global search capability is still limited by the existing tree nodes and topological structure. The introduction of the bisection reconstruction strategy can effectively reduce the number of nodes in the local area and obtain a smooth low-cost path in the local area, which not only reduces the amount of planning calculations, but also increases the speed at which the planner converges to the most efficient path solution.

Algorithm 3: FastExtend.

Input : $X_{target}, T_{init}, Stepsize, \delta$
Output : $X_{new}, T, flag$

- 1 $X_{nearest} \leftarrow \text{Nearest}(V, X_{target});$
- 2 $X_{new} \leftarrow X_{nearest} + Stepsize \cdot (X_{target} - X_{nearest}) / \|X_{target} - X_{nearest}\|;$
- 3 **if** $\text{CollisionFree}(X_{nearest}, X_{new})$ **then**
- 4 $X_{reachest} \leftarrow \text{FindReachest}(T, X_{nearest}, X_{new}, T_{init});$
- 5 $X_{create} \leftarrow \text{CreateNode}(T, X_{reachest}, X_{new}, \delta);$
- 6 **if** $X_{create} \neq \emptyset$ **then**
- 7 $V \leftarrow V \cup \{X_{create}, X_{new}\};$
- 8 $E \leftarrow E \cup \{(\text{Parent}(X_{reachest}), X_{create}), (X_{create}, X_{new})\};$
- 9 **else**
- 10 $V \leftarrow V \cup \{X_{new}\};$
- 11 $E \leftarrow E \cup \{(X_{reachest}, X_{new})\};$
- 12 **end**
- 13 $T \leftarrow (V, E);$
- 14 **return** $(X_{new}, T, flag = \text{Advanced});$
- 15 **else**
- 16 $T \leftarrow (V, E);$
- 17 **return** $(X_{new}, T, flag = \text{Trapped});$
- 18 **end**

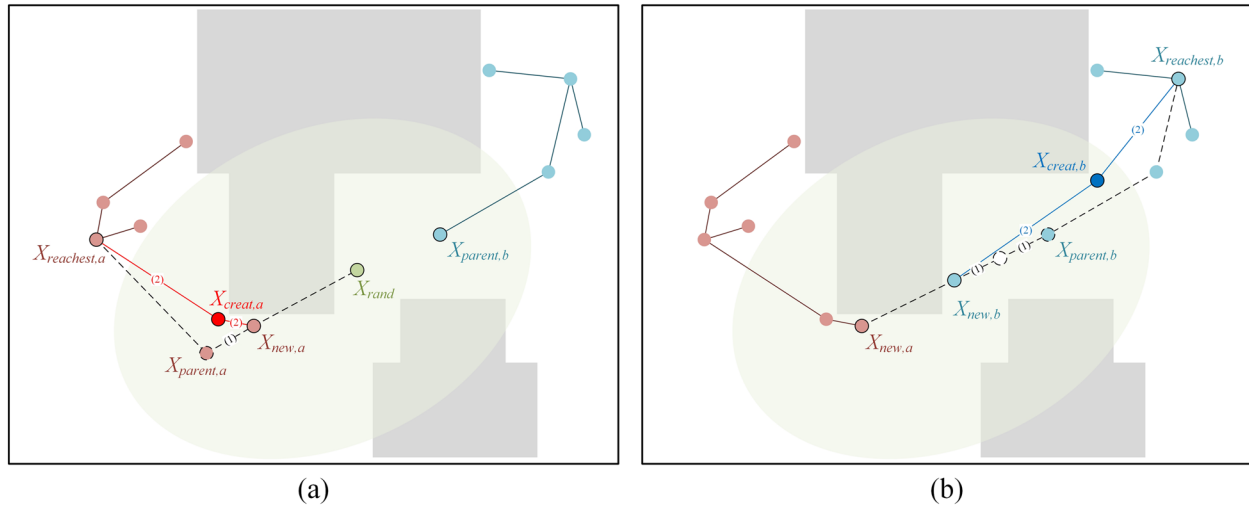


Fig. 3 Fast bidirectional exploration of random trees. (Black dashed lines: original path topology, red and blue solid lines: path topology after bisection optimization, numbers in brackets: order of exploration)

Algorithm 4: FastConnect.

Input : $X_{target}, T_{init}, Stepsize, \delta$
Output : $(X_{new}, T, flag)$

```

1  $X_{nearest} \leftarrow \text{Nearest}(V, X_{target});$ 
2  $X_{new} \leftarrow X_{nearest};$ 
3  $T \leftarrow (V, E);$ 
4 while true do
5    $X_{new} \leftarrow$ 
      $X_{nearest} + Stepsize \cdot (X_{target} - X_{nearest}) / \|X_{target} - X_{nearest}\|;$ 
6   if  $\text{CollisionFree}(X_{nearest}, X_{new})$  then
7      $X_{reachest} \leftarrow \text{FindReachest}(T, X_{nearest}, X_{new}, T_{init});$ 
8      $X_{create} \leftarrow \text{CreateNode}(T, X_{reachest}, X_{new}, \delta);$ 
9     if  $X_{create} \neq \emptyset$  then
10       $V \leftarrow V \cup \{X_{create}, X_{new}\};$ 
11       $E \leftarrow E \cup \{(Parent(X_{reachest}), X_{create}), (X_{create}, X_{new})\};$ 
12    else
13       $V \leftarrow V \cup \{X_{new}\};$ 
14       $E \leftarrow E \cup \{(X_{reachest}, X_{new})\};$ 
15    end
16     $T \leftarrow (V, E);$ 
17    if  $\text{InitialPathFound}(V, X_{target})$  then
18      return  $(X_{new}, T, flag = Reached);$ 
19    end
20     $X_{nearest} \leftarrow X_{new};$ 
21  else
22    break
23  end
24 end
25 return  $(X_{new}, T, flag = Trapped);$ 

```

Algorithm 3 and Algorithm 4 respectively show the process of forward exploration and reverse exploration. Specifically, assume that two trees $T_a = (V_a, E_a)$ and $T_b = (V_b, E_b)$ take the starting point X_{start} and the end point X_{goal} as root nodes respectively. First, expand the forward tree T_a to the sampling point X_{rand} to obtain a new node (line 1–2 of Algorithm 3), and select the nearest neighbor node in the tree node set V_a :

$$X_{nearest} = \arg \min_{X \in V_a} \|X - X_{rand}\| \quad (9)$$

$$X_{reacheast} = \sup \left\{ X \in V_a \mid \overline{X \rightarrow X_{nearest}} \in \mathcal{X}_{free} \wedge X \in \text{Ancestor}(X_{nearest}) \right\} \quad (11)$$

For each path segment $\overline{X_{reacheast} X_{new}}$, the bisection method is used for optimization (line 4 of Algorithm 3), and $X_{forbid} \leftarrow X_{reacheast}$, $X_{connect} \leftarrow X_{new}$, $X_{allow} \leftarrow X_{parent}$ and $X_{forbid} \leftarrow X_{new}$, $X_{connect} \leftarrow X_{forbid}$, $X_{allow} \leftarrow X_{parent}$ are respectively set to converge twice, and the convergence update law is:

$$\begin{cases} X_{mid}^{(k)} = \frac{X_{forbid}^{(k)} + X_{allow}^{(k)}}{2} \\ X_{allow}^{(k+1)} = \begin{cases} X_{mid}^{(k)}, & \text{if } \overline{X_{connect} \rightarrow X_{mid}^{(k)}} \in \mathcal{X}_{free} \\ X_{allow}^{(k)}, & \text{otherwise} \end{cases} \\ X_{forbid}^{(k+1)} = \begin{cases} X_{forbid}^{(k)}, & \text{if } \overline{X_{connect} \rightarrow X_{mid}^{(k)}} \in \mathcal{X}_{free} \\ X_{mid}^{(k)}, & \text{otherwise} \end{cases} \end{cases} \quad (12)$$

The convergence termination law is:

$$\exists K_1 \in \mathbb{N}. \forall k \geq K_1, \|X_{forbid}^{(k)} - X_{allow}^{(k)}\|_2 \leq \delta_{dichotomy} \quad (13)$$

Among them, $\delta_{dichotomy}$ is the accuracy threshold of dichotomy optimization.

The backward expansion process is similar to the forward one, except that the backward exploration is based on the heuristic distance and greedily expands to the new node X_{new} in the tree. The tree will repeatedly perform the expansion operation (Algorithm 4, lines 4–5) until the newly obtained path $\overline{X_{nearest,b} X_{new,b}} \notin \mathcal{X}_{free}$ or satisfies the dual-tree connection condition $\overline{X_{new} X_{new,b}} \in \mathcal{X}_{free}$ (Algorithm 4, lines 6, 17–19).

Therefore, ABF-RRT* achieves multi-scale coordination of spatial exploration and path optimization, and realizes dynamic coupling and balanced optimization of global search and local optimization. Figure 3 shows a complete two-way exploration, where Fig. 3(a) shows the process of forward expansion, and Fig. 3(b) shows the process of reverse greedy expansion based on heuristic distance.

New nodes expanded in a single step:

$$X_{new} = X_{nearest} + \text{Stepsize} \cdot \frac{X_{rand} - X_{nearest}}{\|X_{rand} - X_{nearest}\|} \quad (10)$$

If the generated path $\overline{X_{nearest} X_{new}} \in \mathcal{X}_{free}$, the local path topology is further reconstructed (Algorithm 3, Lines 3–14). As shown in Fig. 3, suppose we backtrack from the nearest neighbor node $X_{nearest}$ to the root node, and find the optimal ancestor node $X_{reacheast}$ of the collision-free link $\overline{X_{nearest}}$ through reverse path verification (Algorithm 3, Line 4), that is:

3.4 Complexity analysis

This section proves that the time and space complexity of ABF-RRT* is the same as that of F-RRT* [29] which also uses the bisection method, and analyzes the key factors affecting the complexity. Time complexity and space complexity are important bases for evaluating the number of resources required by an algorithm during execution and are used to judge the efficiency of an algorithm. Big O Notation is often used to describe how the running time or required space of an algorithm changes when the input data size increases. Given two functions $\mathcal{P}(x)$ and $\mathcal{Q}(x)$, $\mathcal{P}(x)$ is considered to belong to $\mathcal{O}(\mathcal{Q}(x))$ if $\limsup_{x \rightarrow \infty} \left| \frac{\mathcal{P}(x)}{\mathcal{Q}(x)} \right| < \infty$.

3.5 Time complexity

The time complexity is defined as the number of times the time-consuming process is called in the algorithm control flow. The overall time complexity is usually the sum or maximum of the complexity of each stage. Before analyzing ABF-RRT*, let's briefly analyze the time complexity of the F-RRT* [29] algorithm. Assume that in a single iteration, the tree has $\backslash$ nodes. The sampling process *Sample* does not depend on the amount of input data, the expansion process *Steer* only involves interpolation between computing nodes, and the swap process *Swap* is a constant operation, so their time complexity is $\mathcal{O}(1)$.

Nearest, *CollisionFree* and *Rewire* processes need to traverse global parameters (if collision detection is global). The time complexity of these three processes is $\mathcal{O}(\mathcal{S} \cdot \log \backslash)$.

The *FindReacheast* process traces the tree back to the ancestor state $X_{reacheast}$. Let $\mathcal{S}$ be the backtracking depth. The unit depth backtracking requires one call to the *CollisionFree* process, so its time complexity is $\mathcal{O}(\mathcal{S} \cdot \log \backslash)$.

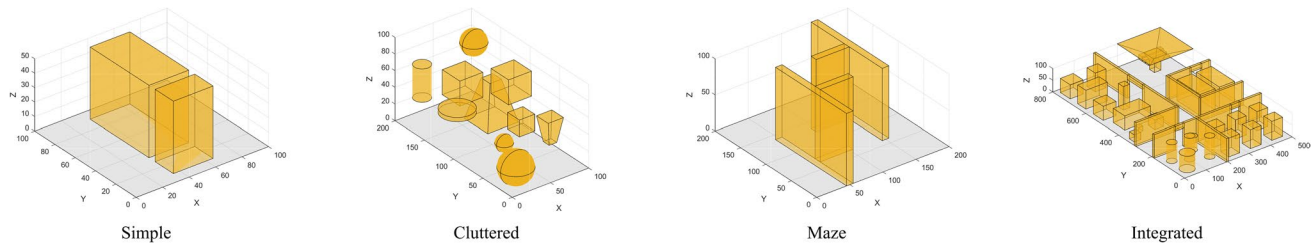


Fig. 4 Four types of test environments

The *CreateNode* process will perform a binary search to find a suitable generation location. Each binary search requires one call to the *CollisionFree* process. Let $\mathcal{D}$ be the distance of the binary search path. In the worst case, the time complexity of the *CreateNode* process is $\mathcal{O}(\log \mathcal{D} \cdot \log \backslash)$.

Therefore, in the case of taking the maximum time complexity, let N be the number of iterations and K be a constant, the overall time complexity of F-RRT* can be expressed as $\mathcal{O}(N \cdot K \cdot (\log \backslash))$.

Next, the time complexity of ABF-RRT* in a single iteration is analyzed. Let $\mathcal{F}^{PRO}$ represent the time complexity of the process PRO.

Lemma 1. $\mathcal{F}^{AdaptiveSample} \in \mathcal{O}(1)$

Proof. Obviously, the *AdaptiveSample* process does not involve traversal or complex calculations, and all of them are simple matrix operations. Therefore $\mathcal{F}^{AdaptiveSample} \in \mathcal{O}(1)$.

Lemma 2. $\mathcal{F}^{FastExtend} \in \mathcal{O}(K \cdot \backslash + \backslash)$

Proof. The *FastExtend* process calls the *Nearest* process once and the *CreateNode* process once. In the worst case, the *CreateNode* process performs binary search on the entire path. Let $\mathcal{C}$ be the number of binary searches and $\mathcal{D}$ be the distance of the binary search path. Then $\mathcal{M}$ and $\mathcal{N}$ satisfy $\forall \mathcal{D} \geq \mathcal{M}, E[|\mathcal{C}|] = \mathcal{N} \cdot \log \mathcal{D}$. So, the complexity of the *FastExtend* process is $\mathcal{O}(\log \mathcal{D} \cdot \backslash + \backslash)$. Therefore, $\mathcal{F}^{FastExtend} \in \mathcal{O}(K \cdot \backslash + \backslash)$.

Lemma 3. $\mathcal{F}^{FastConnect} \in \mathcal{O}(K \cdot \log \backslash + \backslash)$

Proof. The *FastConnect* process first calls the *Nearest* process once, and then calls the *CreateNode* process once for each step in the greedy exploration. The corresponding complexity is $\mathcal{O}(\log \mathcal{D} \cdot \log \backslash)$. So, the time complexity of the *FastConnect* process is $\mathcal{O}(\log \mathcal{D} \cdot \log \backslash + \backslash)$. Therefore, $\mathcal{F}^{FastConnect} \in \mathcal{O}(K \cdot \log \backslash + \backslash)$. In the case of taking the maximum time complexity, let N be the number of iterations and K is a constant. The overall time complexity of ABF-RRT* can be expressed as $\mathcal{O}(N \cdot K \cdot (\log \backslash))$.

3.6 Space complexity

Space complexity is defined as the growth of storage resources used by the algorithm during its operation. In RRT algorithms, the tree $T = (V, E)$ determines the size of the memory space. Let $T_N = (V_N, E_N)$ represent the random tree after N times.

Lemma 4. $|T_N| \in \mathcal{O}(\backslash)$

Proof. Assume that at N iterations, the tree has $\backslash$ nodes. Then the number of nodes $|V_N| = n$, the number of edges $|E_N| = n - 1$, that is, $|T_N| = 2n - 1$. Obviously, the space complexity of F-RRT* [29] and ABF-RRT* satisfies $|T_N| \in \mathcal{O}(\backslash)$.

3.7 Analysis of influencing factors

In the above analysis, due to the presence of parameter thresholds, the variation in the constant K is relatively small. At the same time, the number of nodes $\backslash$ and the number of

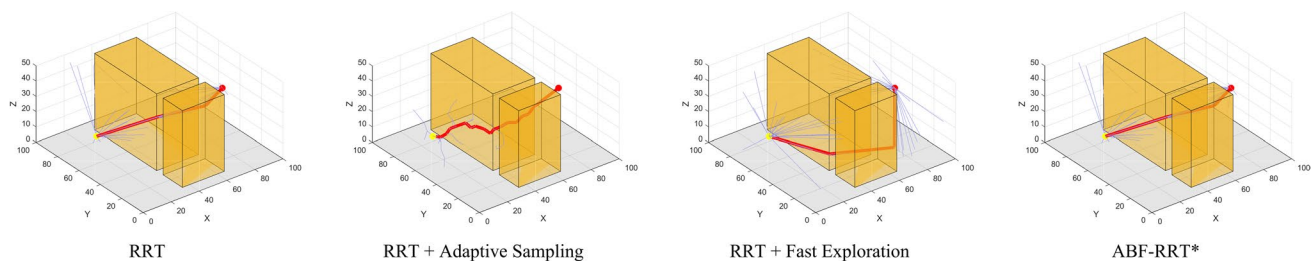


Fig. 5 Validation of the core method in a simple environment (light blue: topology of the random tree, red: final planned path, yellow dot: starting point, red dot: target point)

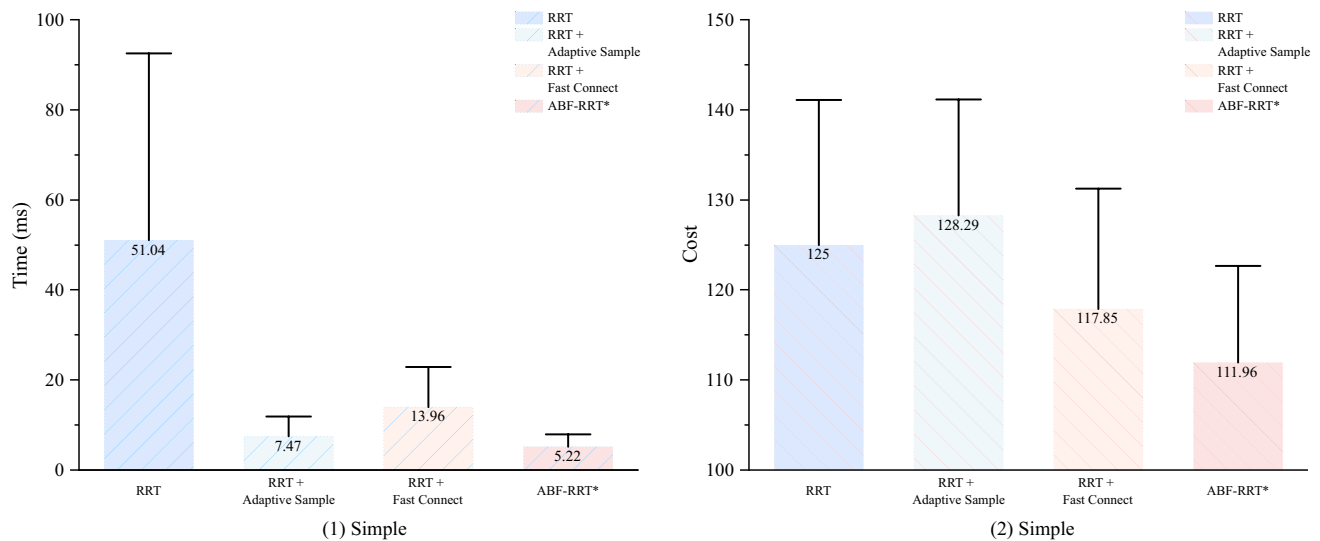


Fig. 6 Planning data for a simple environment, where the marked data are the average values and the black line is the standard deviation: (1) planning time, (2) path cost

Table 1 Comparison data of average path planning performance between the original RRT and the two core methods under different combination configurations

Algorithm	Avg T_{init} (ms)	Avg C_{init}	Avg N_{init}	S_{init}
RRT	51.04	125	83.4	1
RRT + Adaptive Sampling	7.46	128.29	72.32	1
RRT + Fast Connect	13.96	117.85	68.14	1
ABF-RRT*	5.22	111.96	54.8	1

iterations N are positively correlated. Therefore, the number of iterations N is a key parameter affecting the performance of RRT algorithms. The increase in N will lead to a surge in computing time during the planning process. Therefore, reducing redundant sampling and expansion is the key to ensuring the efficiency of ABF-RRT* algorithms.

4 Simulation results and analysis

This section first integrates the new sampler and planning cycle into the baseline algorithm RRT, and verifies the effectiveness and advantages of its core improvement method on the basic map Simple. Then, the proposed ABF-RRT algorithm is compared and analyzed with several existing representative algorithms. The comparison algorithms include: JPS based on graph search, APF based on potential field, ACO based on heuristics, and Q-RRT*, F-RRT and ABQ-RRT based on sampling. To achieve a comprehensive evaluation, two types of test environments are designed: basic environment and integrated environment. The basic environment covers typical scenarios, such as cluttered areas

(Cluttered) and mazes (Maze); the integrated environment is an indoor map containing multiple rooms, corridors with various types of obstacles, and an open area. Figure 4 shows all the test maps.

Each comparison process will be completed through two simulation experiments. The first experiment uses four evaluation indicators to compare the quality of the initial path generated by the algorithm: T_{init} is the time to obtain the initial solution, C_{init} is the Euclidean distance cost of the initial solution, N_{init} is the number of nodes used, and S_{init} is the planning success rate. The second experiment uses three indicators to evaluate the efficiency of the algorithm converging to the optimal solution based on the initial solution: $T_{optimal}$ is the time for the path cost to converge to '1.05 * Optimal Cost', $N_{optimal}$ is the number of nodes used, and $S_{optimal}$ is the convergence success rate. It should be noted that due to the fixed strategy of the graph search and potential field algorithms, their initial path costs are constant and do not have convergence optimization capabilities; only the sampling-based algorithm supports node expansion and optimal solution convergence processes.

The simulation experiment was conducted on a machine equipped with an Intel i7-11800H 2.30GHz processor and 16GB RAM, and the simulation platform was MATLAB R2022b. The maximum number of planning iterations was 20,000, and the time unit was milliseconds in the cluttered and maze environments and seconds in the comprehensive environment. If no path was found within the specified number of iterations or the planning time exceeded 60 s, the planning was considered to have failed. In the experiment, the exploration step size was set to 5. In APF, the attraction coefficient was 1.0, the repulsion coefficient was 120.0, the repulsion distance was 6, and the disturbance amplitude was 0.5. In ACO,

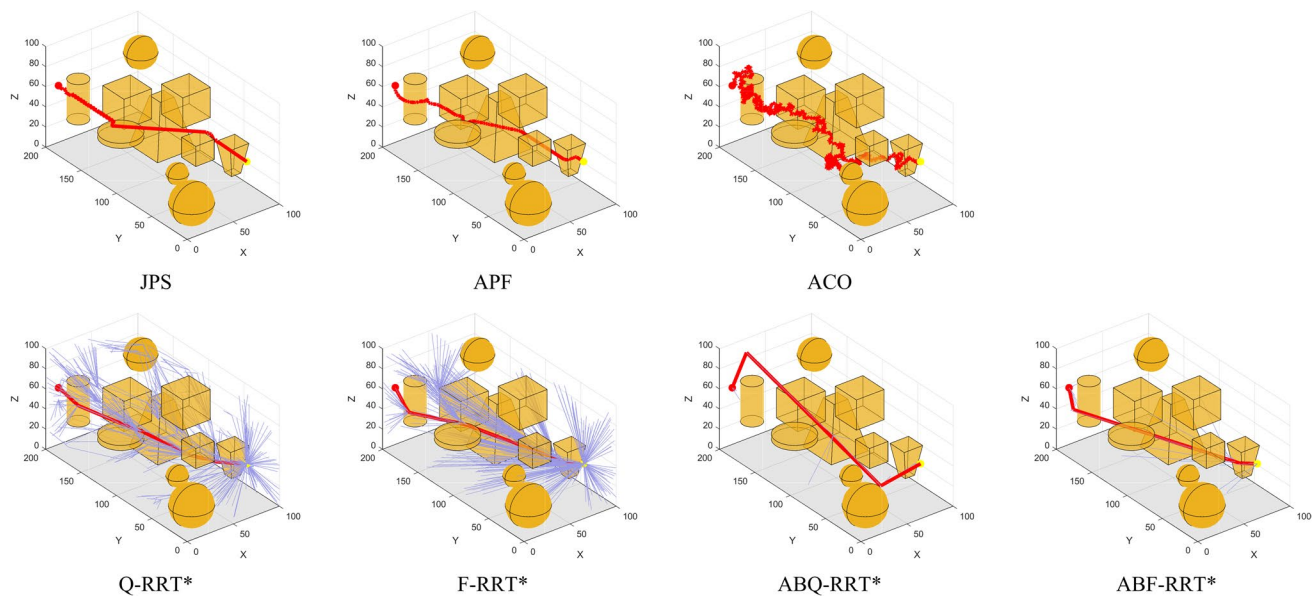


Fig. 7 Comparison results in a cluttered environment (light blue: topology of the random tree, red: final planned path, yellow dot: starting point, red dot: target point)

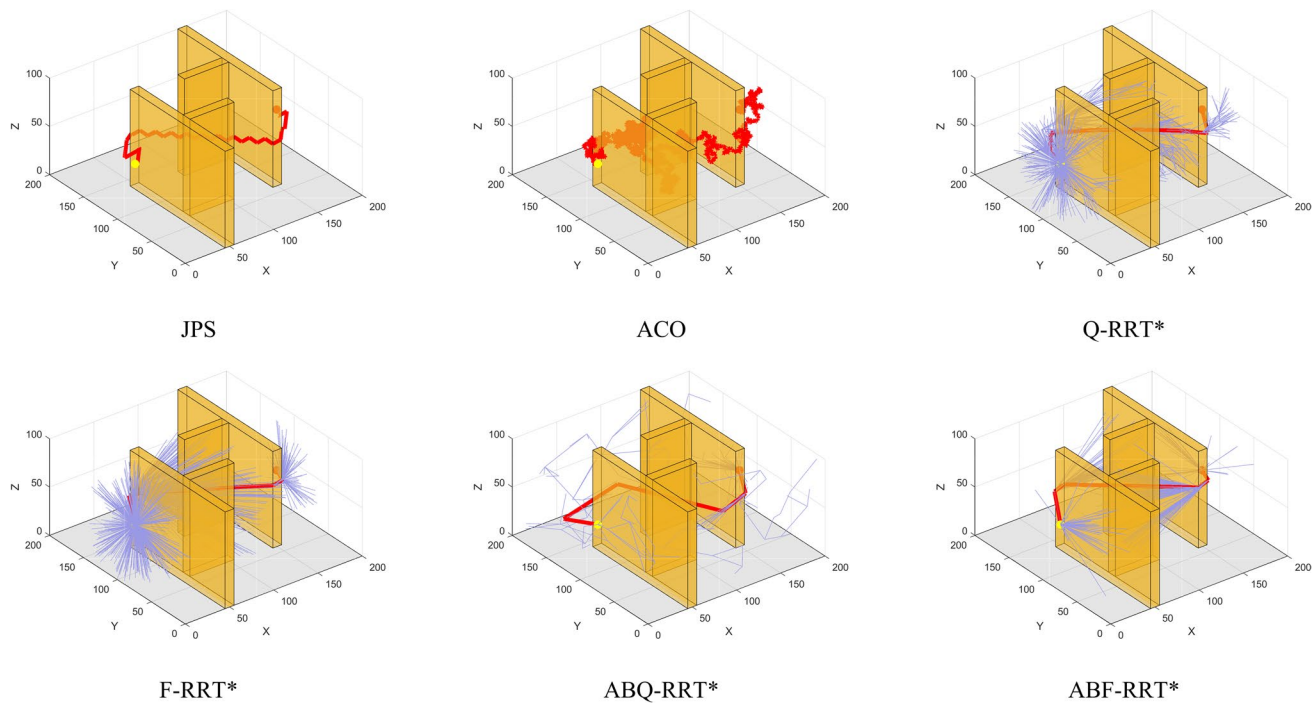


Fig. 8 Comparison results in a maze environment (light blue: topology of the random tree, red: final planned path, yellow dot: starting point, red dot: target point)

the number of ants was 30, the pheromone volatility was 0.1, the pheromone factor was 1, and the heuristic factor was 3. In the RRT algorithm, the hypersphere radius was set to 10 times the step size, the dichotomy optimization threshold $\delta_{dichotomy}$

was set to 2, the sampling probability p_{init} in the adaptive hyperellipsoid in the initial stage was set to 0.5, and the hyperellipsoid parameter α was set to 0.5. Statistics are from 50 successful independent runs of each algorithm.

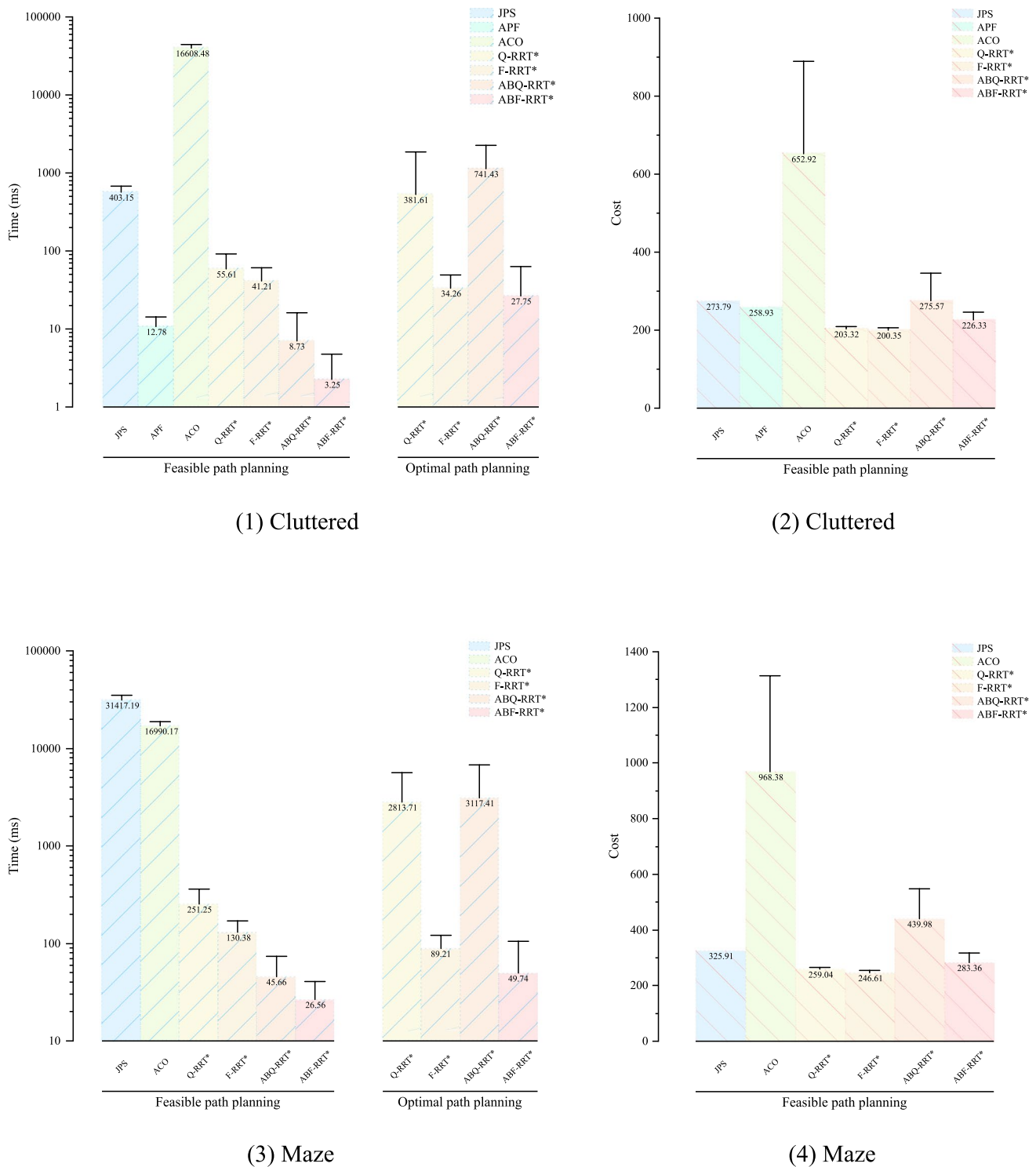


Fig. 9 Planning data in basic environments, where the labeled data is the mean and the black line is the standard deviation: (1) planning time in a cluttered environment, (2) path cost in a cluttered environment, (3) planning time in a maze environment, (4) path cost in a maze environment

4.1 Core method validation

The plans of the four sampler and planner combinations are shown in Fig. 5, and Fig. 6 shows the time statistics of the

plans. Obviously, compared with the baseline algorithm RRT, the new sampler significantly reduces the redundant exploration in space. It not only concentrates most of the sampling points in the key area of the planned path, but also uses a small

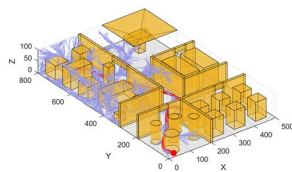
number of sampling points to take into account the exploration of other areas. At the same time, the new bidirectional planner obtains a smoother and lower-cost path than RRT. ABF-RRT*, the combination of the new sampler and the new planner, achieves better results than a single combination.

Table 1 shows the average data of planning. The combination of the new sampler and the new planner uses only the least time and nodes to obtain the path with the best cost. Compared with the baseline algorithm RRT, the new sampler saves 85.38% of the average planning time and 13.29% of the

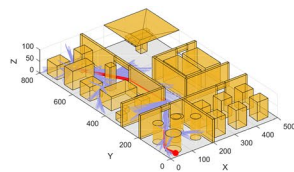
average number of nodes, but the average path cost increases by 2.63%. The new planner saves 72.65% of the average planning time, 5.72% of the average path cost and 18.30% of the average number of nodes. The combination of the two saves 89.77% of the average planning time, 10.43% of the average path cost and 34.29% of the average number of nodes. This not only proves the individual effectiveness and superiority of the two core methods, but also highlights the strong coupling and strong complementarity between the two. That is, the new sampler provides the planner with key information about the

Table 2 Comparison data of average path planning performance of seven algorithms under Basic environment

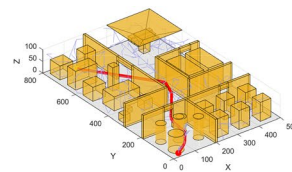
Scenario	Algorithm	Feasible path planning				Optimal path planning		
		Avg T_{init} (ms)	Avg C_{init}	Avg N_{init}	S_{init}	Avg $T_{optimal}$ (ms)	Avg $N_{optimal}$	$S_{optimal}$
Cluttered	JPS	403.15	273.79	/	1	/	/	/
	APF	12.78	258.93	/	1	/	/	/
	ACO	16608.48	652.92	/	1	/	/	/
	Q-RRT*	55.61	203.32	437.64	1	381.61	1060.92	1
	F-RRT*	41.21	200.35	791.98	1	34.26	4996.12	0.8
	ABQ-RRT*	8.73	275.57	24.12	1	741.43	1915.96	1
	ABF-RRT*	3.25	226.33	31.7	1	27.75	365.26	1
Maze	JPS	31417.19	325.91	/	1	/	/	/
	APF	NA	NA	/	0	/	/	/
	ACO	16990.17	968.38	/	0.74	/	/	/
	Q-RRT*	251.25	259.04	1046.6	1	2813.71	3611.54	1
	F-RRT*	130.38	246.61	1847.56	1	89.21	1621.89	0.92
	ABQ-RRT*	45.66	439.98	150.42	1	3117.41	5719.98	1
	ABF-RRT*	26.56	283.36	367.28	1	49.74	709.84	1



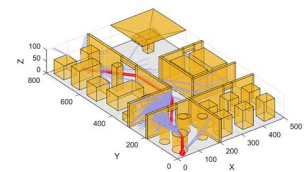
Q-RRT*



F-RRT*



ABQ-RRT*



ABF-RRT*

Fig. 10 Comparison results in scenario I in an integrated environment (light blue: topology of the random tree, red: final planned path, yellow dot: starting point, red dot: target point)

Table 3 Comparison data of average path planning performance of seven algorithms in Integrated environment

Scenario	Algorithm	Feasible path planning				Optimal path planning		
		Avg T_{init} (s)	Avg C_{init}	Avg N_{init}	S_{init}	Avg $T_{optimal}$ (s)	Avg $N_{optimal}$	$S_{optimal}$
Integrated I	Q-RRT*	2.41	945.54	6032.70	0.88	14.08	9032.33	0.18
	F-RRT*	1.62	862.29	7760.17	0.92	1.82	6881.56	0.72
	ABQ-RRT*	0.22	1404.72	533.48	1	7.00	6895.02	1
	ABF-RRT*	0.11	907.44	229.98	1	0.92	3581.98	1
Integrated II	Q-RRT*	NA	NA	NA	0	NA	NA	0
	F-RRT*	NA	NA	NA	0	NA	NA	0
	ABQ-RRT*	0.73	1031.81	363.82	1	3.50	6101.84	1
	ABF-RRT*	0.48	751.75	2588.06	1	1.53	6511.82	1

environment through efficient sampling. The new planner can make full use of the adopted information to perform accurate and efficient path search and optimization. The two synergies significantly improve the efficiency of path planning.

4.2 Basic environment

Cluttered and maze environments are widely used path planning test scenarios. In cluttered environments, the increase

in the distance between states leads to a decrease in direct connectivity. Therefore, the performance of the algorithm not only depends on the exploration ability, but also needs to efficiently avoid redundant exploration and collisions; sparse corridor environments can effectively test the ability of the algorithm to converge to the optimal path.

Figures 7 and 8 show the path planning results of the four algorithms. JPS still has room for optimization in terms of path cost; APF is prone to fall into local minima in maze

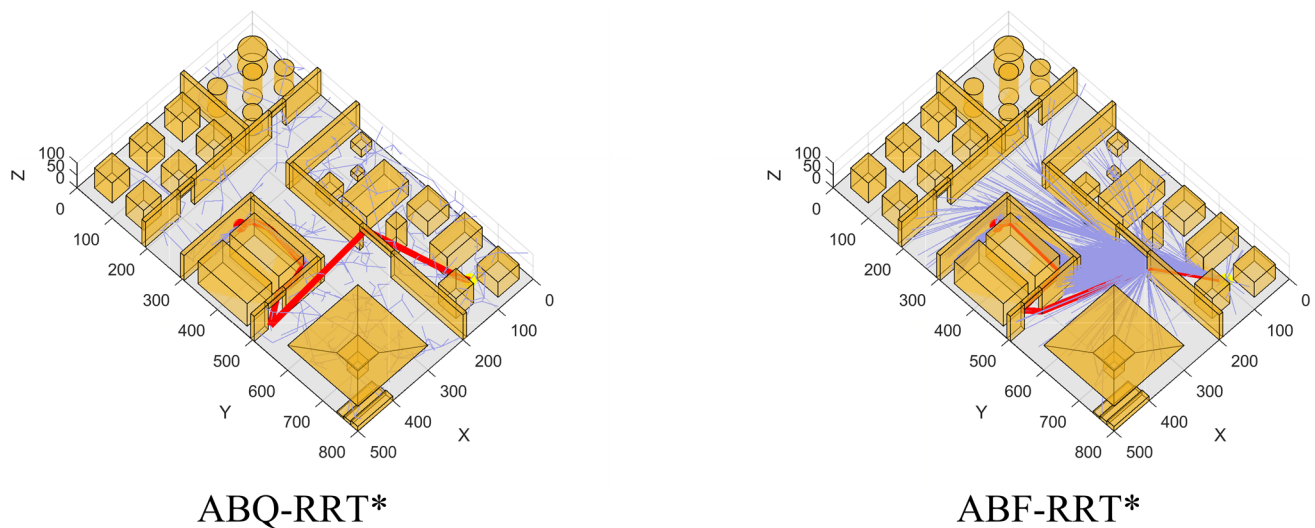


Fig. 11 Comparison results in scenario II in an integrated environment (light blue: topology of the random tree, red: final planned path, yellow dot: starting point, red dot: target point)

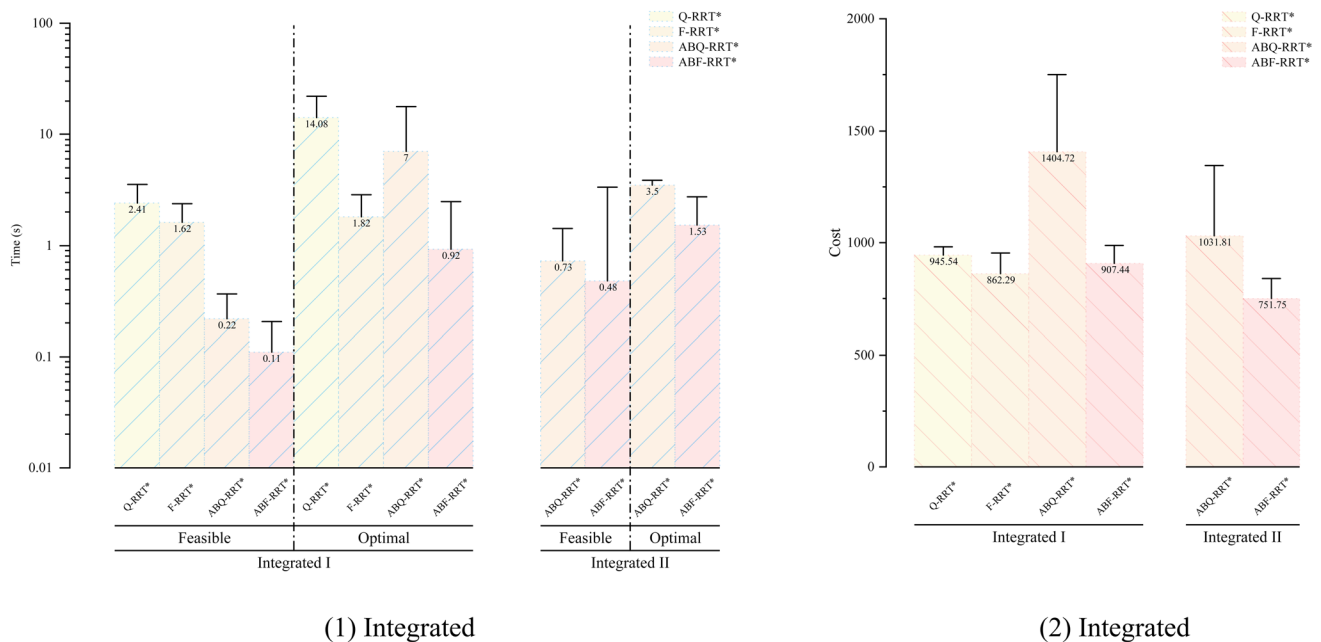


Fig. 12 Planning data in an integrated environment, where the labeled data is the mean and the black line is the standard deviation: (1) planning time (2) path cost

environments. Although it can complete global planning in complex environments, its performance is unstable. ACO can find feasible paths, but it is difficult to generate smooth paths within a limited number of iterations or time. In contrast, the paths generated by the four improved algorithms based on RRT are smoother. Among them, ABQ-RRT* can complete the exploration with fewer nodes through the adaptive bias sampling strategy, but the path cost is higher; Q-RRT* and F-RRT* have similar performance, the latter combines bisection optimization, the path quality is better but requires more nodes; ABF-RRT* converges to the optimal path equivalent to F-RRT* with fewer nodes.

The data in Fig. 9 further show that ABF-RRT* performs best in both initial solution generation and optimal solution convergence time. The planning time of JPS and ACO in a three-dimensional environment is significantly higher than that of other algorithms, due to the exponential computational growth caused by the dimensional expansion (Stentz 1994; Botea et al. 2002). Table 2 summarizes the average indicators. In a chaotic environment, the average time for ABF-RRT* to obtain the initial solution is only 0.81%, 25.43%, 0.2%, 5.84%, 7.89% and 37.22% of JPS, APF, ACO, Q-RRT*, F-RRT* and ABQ-RRT*, and the time to converge to the optimal solution is 7.27%, 81.00% and 3.74% of the other RRT algorithms. In a maze environment, its initial solution time is 0.8%, 0.15%, 10.57%, 20.37% and 58.17% of other successful algorithms, and its convergence time is 1.77%, 55.76% and 1.60% of other RRT algorithms, which verifies its advantages in fast generation of initial paths and efficient convergence.

Although ABF-RRT* has a slightly higher cost than Q-RRT* and F-RRT* in the initial solution stage, it can quickly converge to the optimal path with the minimum cost during the overall optimization process. Although ABQ-RRT* can quickly generate the initial path, due to its reliance on local sampling around obstacles, the path cost is high and the convergence efficiency is low. It is worth noting that the average time for the optimal solution of F-RRT* is lower than that of the initial solution, mainly because its optimal convergence success rate is 92%, while the initial solution is only 80%, and some failure cases are not included in the average. Overall, ABF-RRT* achieved a 100% planning success rate in all tasks, with the best overall performance.

4.3 Integrated environment

The integrated environment simulates an indoor scene consisting of multiple rooms connected by a single corridor, aiming to test the planning ability of the algorithm in a complex high-dimensional environment. The test includes two simulation scenes, and the planning of both scenes requires traversing multiple basic environments. Among them, the

target state of scenario II is set in a crowded room. Due to the thin walls of the room and the narrow interior space, the planner is prone to repeatedly optimize on a fixed path instead of bypassing narrow obstacles, that is, falling into a local optimal solution.

The path results of scenario I are shown in Fig. 10. Only the RRT class algorithm completed the planning task, which once again proves its excellent robustness in high-dimensional environments. Table 3 and Fig. 12 present the statistical results of the RRT algorithm, as all other algorithms failed to generate a feasible path. Q-RRT* takes the longest time and has the lowest success rate in the two planning tasks, indicating that it cannot solve the planning problem in the synthetic environment well. Although F-RRT* can plan the path with the lowest initial cost, its initial planning time is long, and the success rate in both tasks is not 100%. The initial planning time of ABQ-RRT* is significantly improved compared with Q-RRT* and F-RRT*, but the obtained path cost is high and the convergence speed is slow. In contrast, ABF-RRT* showed the best time performance in both initial path planning and optimal path planning. The planning speed of its initial solution was about 22 times, 15 times and 2 times that of Q-RRT*, F-RRT* and ABQ-RRT*, and the convergence speed was about 15 times, 2 times and 8 times that of Q-RRT*, F-RRT* and ABQ-RRT*, which proved the robustness and adaptability of the new algorithm.

The path results of scenario II are shown in Fig. 11. Only ABQ-RRT* and ABF-RRT* can complete the planning task, which once again proves the superiority of heuristic bidirectional exploration. The statistical results of the RRT algorithm are still shown in Table 3 and Fig. 12, as all other algorithms failed to generate a feasible path. Both algorithms maintained a 100% success rate. Among them, ABQ-RRT* benefits from the paranoid sampling of the edge of obstacles in narrow areas and uses fewer nodes, but this still leads to a large path cost, and because its planning relies too much on the nodes at the edge of narrow channels, it is difficult to converge to the optimal solution quickly. In contrast, the planning time and initial path cost of ABF-RRT* are 65.75% and 72.86% of those of ABQ-RRT*, respectively, and the convergence speed is about twice that of ABQ-RRT*. This once again proves the adaptability of the new algorithm and that its planning performance in normal environments is not significantly affected by the additional calculations required to handle more obstacles.

5 Discussion

There are two new designs in ABF-RRT*, one is a new adaptive sampler, and the other is a heuristic bidirectional exploration coupled with dichotomy. The new sampler is inspired by Informed-RRT*, but there are essential differences. The

strategy of Informed-RRT* is to frame a hyper-ellipsoid based on the initial solution, and then accelerate the convergence to the optimal solution by rejecting the nodes outside the hyper-ellipsoid. In contrast, the new sampler of ABF-RRT* is applied to both the initial solution planning and the optimal solution planning process, and effectively balances the global expansion and local exploration in the planning process through the probability formula (2) and the adaptive hyper-ellipsoid. At the initial moment of planning, a relatively balanced sampling is used to allow the planner to better explore more space outward. As the sampling proceeds, the sampler gradually shrinks the hyper-ellipsoid. On the one hand, it accelerates the advancement of the dual tree through more concentrated sampling, and on the other hand, it retains a moderate amount of sampling outside the hyper-ellipsoid to avoid falling into the local optimum. The new planner couples the strategy of creating alternative optimal nodes by dichotomy into the bidirectional heuristic exploration, and sets a new convergence update law to make up for the defects of high path cost and slow convergence speed of bidirectional exploration.

As a global path planning algorithm, ABF-RRT* is only applicable to path planning in static obstacle environments. When the actual robot performs trajectory tracking, it is necessary to combine the local path planning algorithm with the original path solution to avoid obstacles. We are trying to shift the workspace of ABF-RRT* from static to dynamic or uncertain environments, and build an algorithm framework that can respond to dynamic environments in real time, similar to CL-RRT* (Luders et al. 2010), RT-RRT* (Naderi et al. 2015) and CDRT-RRT* (Liu et al. 2025). Alternatively, the path planned by ABF-RRT* can be discretized into a series of reference points, and then combined with the optimal control strategy to complete the real-time control of the robot, similar to the strategy combined with model predictive control (MPC) application in DQN-MPC (Zhang et al. 2023) and DRL-MPC (Ceder et al. 2024).

6 Conclusion

This paper proposes an optimal path planning algorithm ABF-RRT*, which combines a new adaptive sampler with a fast-converging bidirectional exploration strategy. The sampler can dynamically adjust the sampling strategy according to the exploration progress of the dual tree to balance the utilization and exploration in the sampling process. The exploration strategy retains the beneficial properties of bisection optimization and couples it to heuristic-based bidirectional exploration. This paper verifies the effectiveness and superiority of the proposed core method in a basic environment, and compares the performance of existing algorithms and

new algorithms in a variety of different environments. The results show that the proposed APF-RRT algorithm can effectively solve various planning problems. In addition, since the proposed adaptive sampler and bidirectional exploration strategy are both tree expansion algorithms, they can be combined with any other sampler or graph pruning strategy. We also discuss the key factors influencing the efficiency of the algorithm and analyze the limitations of the ABF-RRT algorithm, thereby outlining potential directions for future research in this area.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1007/s44443-025-00186-0>.

Acknowledgements This work was supported by the National Natural Science Foundation of China [grant number 62473133].

Author contributions Zeng Zhaoyang: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing—original draft, Writing—review & editing. Chang Yufang: Methodology, Validation, Writing—review & editing, Supervision, Funding acquisition, Project administration.

Data availability Data will be made available on request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Alabbadi A, Kanan A (2023) Genetic algorithm-based path planning for autonomous mobile robots. In: Proc IEEE Jordan Int Joint Conf Electr Eng Inf Technol (JEEIT), pp. 177–180. <https://doi.org/10.1109/JEEIT58638.2023.10185855>
- Arslan O, Tsiotras P (2013) Use of relaxation methods in sampling-based algorithms for optimal motion planning. In: Proc IEEE Int Conf Robot Autom
- Boscariol P, Rossi R, Fiorini P (2015) Path planning and trajectory planning algorithms: a general overview. Mech Mach Sci 36:25–38. https://doi.org/10.1007/978-3-319-14705-5_1

- Botea A, Müller M, Schaeffer J (2002) Using abstraction for planning in Sokoban. In: van den Herik HJ, Iida H, Heinz EA (eds) *Computers and Games CG 2002*, LNCS, vol 2883. Springer, Berlin, pp 360–375. https://doi.org/10.1007/978-3-540-40031-8_24
- Ceder K, He Y, Vu D, Zhang B, Lee D, Ahn HS (2024) Bird's-eye-view trajectory planning of multiple robots using continuous deep reinforcement learning and model predictive control. In: *Proceedings of the 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Abu Dhabi, United Arab Emirates, pp. 8002–8008. <https://doi.org/10.1109/IROS58592.2024.10801434>
- Dijkstra EW (1959) A note on two problems in connexion with graphs. *Numer Math* 1:269–271. <https://doi.org/10.1007/BF01386390>
- Fox D, Burgard W, Thrun S (1997) The dynamic window approach to collision avoidance. *IEEE Robot Autom Mag* 4(1):23–33. <https://doi.org/10.1109/100.580977>
- Gammell JD, Srinivasa SS, Barfoot TD (2014a) Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In: *IEEE/RSJ Int Conf Intell Robot Syst*, pp. 2997–3004. <https://doi.org/10.1109/IROS.2014.6942949>
- Gammell JD, Srinivasa SS, Barfoot TD (2014b) Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs. *arXiv preprint arXiv:1405.5848*. <https://arxiv.org/abs/1405.5848>
- Hart PE, Nilsson NJ, Raphael B (1968) A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans Syst Sci Cybern* 4(2):100–107. <https://doi.org/10.1109/TSSC.1968.300136>
- Huang T, Fan K, Sun W, Li W, Guo H (2023) Potential-Field-RRT: A path-planning algorithm for UAVs based on potential-field-oriented greedy strategy to extend random tree. *Drones* 7(5):331. <https://doi.org/10.3390/drones7050331>
- Huang Z, Gao Y, Guo J, Qian C, Chen Q (2024) An adaptive bidirectional quick optimal Rapidly-exploring Random Tree algorithm for path planning. *Eng Appl Artif Intell* 135:108776. <https://doi.org/10.1016/j.engappai.2024.108776>
- Islam F, Nasir J, Malik U, Ayaz Y, Hasan O (2012) RRT*-Smart: Rapid convergence implementation of RRT* towards optimal solution. In: *IEEE Int Conf Mechatron Autom*, pp. 1651–1656. <https://doi.org/10.1109/ICMA.2012.6359484>
- Jeong I-B, Lee S-J, Kim J-H (2019) Quick-RRT*: Triangular inequality-based implementation of RRT* with improved initial solution and convergence rate. *Expert Syst Appl* 123:82–90. <https://doi.org/10.1016/j.eswa.2019.01.020>
- Jordan M, Perez A (2013) Optimal bidirectional rapidly-exploring random trees. *MIT-CSAIL-TR-2013-021*
- Karaman S, Walter MR, Perez A, Frazzoli E, Teller S (2011) Any-time motion planning using the RRT*. In: *Proc IEEE Int Conf Robot Autom*, pp. 1478–1483
- Karaman S, Frazzoli E (2011) Sampling-based algorithms for optimal motion planning. *Int J Robot Res* 30(7):846–894. <https://doi.org/10.1177/0278364911406761>
- Khatib O (1985) Real-time obstacle avoidance for manipulators and mobile robots. In: *Proceedings of the 1985 IEEE International Conference on Robotics and Automation*, pp. 500–505. IEEE, St. Louis, MO, USA. <https://doi.org/10.1109/ROBOT.1985.1087247>
- Kim D, Lee J, Yoon S-E (2014) Cloud RRT*: Sampling cloud-based RRT*. In: *IEEE Int Conf Robot Autom*, pp. 2519–2526. <https://doi.org/10.1109/ICRA.2014.6907113>
- Koenig S, Likhachev M, Furcy D (2004) Lifelong planning A*. *Artif Intell* 155(1–2):93–146
- Kuffner JJ, LaValle SM (2000) RRT-connect: An efficient approach to single-query path planning. *IEEE Trans Robot Autom* 16(6):793–803. <https://doi.org/10.1109/70.895825>
- LaValle SM (1998) Rapidly-exploring random trees: A new tool for path planning. *IEEE Trans Robot Autom* 14(1):443–449. <https://doi.org/10.1109/70.700224>
- LaValle SM (2006) *Planning Algorithms*. Cambridge University Press, Cambridge
- Li B, Chen B (2022) An adaptive rapidly-exploring random tree. *IEEE/CAA J Autom Sin* 9(2):283–294. <https://doi.org/10.1109/JAS.2022.9673667>
- Li Y, Wei W, Gao Y, Wang D, Fan Z (2020) PQ-RRT*: An improved path planning algorithm for mobile robots. *Expert Syst Appl* 152:113425. <https://doi.org/10.1016/j.eswa.2020.113425>
- Li J, Xiong Y, She J (2021) An improved ant colony optimization for path planning with multiple UAVs. In: *Proc IEEE Int Conf Mechatronics (ICM)*, pp. 1–5. <https://doi.org/10.1109/ICM46511.2021.9385695>
- Liao B, Wan F, Hua Y, Ma R, Zhu S, Qing X (2021) F-RRT*: An improved path planning algorithm with improved initial solution and convergence rate. *Expert Syst Appl* 184:115457. <https://doi.org/10.1016/j.eswa.2021.115457>
- Lin Y, Zhang W, Mu C, Wang J (2022) Application of improved RRT algorithm in unmanned surface vehicle path planning. In: *Proc Chin Control Decis Conf (CCDC)*, pp. 4861–4865. <https://doi.org/10.1109/CCDC55256.2022.10034282>
- Liu J, Fu M, Zhang W, Chen B, Sychou U, Belotserkovsky A (2025) CDRT-RRT*: Real-time rapidly exploring Random Tree Star based on convex dissection. *Expert Syst Appl* 268:126291. <https://doi.org/10.1016/j.eswa.2024.126291>
- Luders BD, Karaman S, Frazzoli E, How JP (2010) Bounds on tracking error using closed-loop rapidly-exploring random trees. In: *Proc Am Control Conf*, pp. 5406–5412
- Naderi K, Rajamäki J, Hämäläinen P (2015) RT-RRT*: A real-time path planning algorithm based on RRT. In: *Proc 8th ACM SIGGRAPH Conf Motion Games*, pp. 113–118
- Shami TM, El-Saleh AA, Alswaitti M, Al-Tashi Q, Summakieh MA, Mirjalili S (2022) Particle swarm optimization: A comprehensive survey. *IEEE Access* 10:10031–10061. <https://doi.org/10.1109/ACCESS.2022.3142859>
- Siciliano B, Khatib O (eds) (2016) *Springer Handbook of Robotics*, 2nd edn. Springer, Berlin
- Sidi M (1997) *Spacecraft dynamics and control: A practical engineering approach*. Cambridge University Press, Cambridge
- Stentz A (1994) Optimal and efficient path planning for partially-known environments. In: *Proc IEEE Int Conf Robot Autom*, 3310–3317. <https://doi.org/10.1109/ROBOT.1994.351061>
- Tahirovic A, Ferizbegovic M (2018) Rapidly-exploring random vines (RRV) for motion planning in configuration spaces with narrow passages. In: *Proc IEEE Int Conf Robot Autom*, pp. 7055–7062. <https://doi.org/10.1109/ICRA.2018.8460186>
- Tong Y, Wu H, Zheng X, Chen Y, Chen Z (2021) Fast jump point search based path planning for mobile robots. In: *Proc 4th Int Conf Intell Auton Syst (ICoIAS)*, pp. 81–85. <https://doi.org/10.1109/ICoIAS53694.2021.00081>
- Wang J, Chi W, Li C, Wang C, Meng MQH (2020) Neural RRT*: Learning-based optimal path planning. *IEEE Trans Autom Sci Eng*. <https://doi.org/10.1109/TASE.2020.2999218>
- Wang J, Chi W, Li C, Meng MQH (2022) Efficient robot motion planning using bidirectional-unidirectional RRT extend function. *IEEE Trans Autom Sci Eng* 19(3):1859–1868. <https://doi.org/10.1109/TASE.2021.3130372>

- Xing Z, Meng Z, Zheng G, Ma G, Yang L, Guo X, Tan L, Jiang Y, Wu H (2025a) Intelligent rehabilitation in an aging population: empowering human-machine interaction for hand function rehabilitation through 3D deep learning and point cloud. *Front Comput Neurosci* 19:1543643. <https://doi.org/10.3389/fncom.2025.1543643>
- Xing Z, Ma G, Wang L, Yang L, Guo X, Chen S (2025b) Towards visual interaction: Hand segmentation by combining 3D graph deep learning and laser point cloud for intelligent rehabilitation. *IEEE Internet Things J* 3546874. <https://doi.org/10.1109/JIOT.2025.3546874>
- Yoshikawa T, Arimoto S (1994) A general formulation of under-actuated manipulator systems. *IFIP Adv Inf Commun Technol* 118:45–56. https://doi.org/10.1007/978-1-4471-1580-9_4
- Zhang Z, Wang D, Guo Q, He Y, Zhang B (2023) Collision-free trajectory planning of mobile robots by integrating deep reinforcement learning and model predictive control. In: *Proceedings of the 2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, Auckland, New Zealand, pp. 1–7. <https://doi.org/10.1109/CASE56687.2023.10260515>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.